

DBMS Sessional Exam: Complete Study Notes

Nirma University, MCA Semester I | Database Management System | Sessional units only

These notes follow your sessional syllabus PDF and the priority list from the past-paper analysis (Jan-22, Feb-22, Jan-23, Jan-25). Every topic carries a priority tag, a simple explanation, and an example.

How to Read These Notes

Tag	Meaning	What to do
[🔥 HIGH-YIELD: Frequently Asked]	Asked in 2 or more past papers, or carries 8-9 marks	Learn fully. Practise writing the answer from memory.
[⭐ ESSENTIAL FOR EXAM]	Asked at least once, or closely linked to a high-yield topic	Learn the definition, one example, and one diagram.
[💡 POSSIBLE SURPRISE: In Syllabus, Not Yet Asked]	In your syllabus but never asked so far	Read once; know the definition and one example.
[📖 FOUNDATION: Background Concept]	Needed to understand the tagged topics	Read it; it makes the rest easy.

Note: All diagrams are ASCII art in a fixed-width font so they stay aligned when you copy them into Word. Keep the Courier New font for diagram blocks.

The Five Topics That Earn Most Marks

Rank	Topic	Where in these notes
1	CREATE TABLE and ALTER TABLE	Unit 3, Sections 3.3 and 3.4
2	Normalization (1NF to 3NF)	Unit 4
3	SQL Keys and Referential Integrity	Unit 2, Sections 2.3 and 2.4
4	Joins and Set Operations	Unit 3, Sections 3.6 and 3.7
5	DBA roles and functions	Unit 1, Section 1.9

Exam Tip: Almost every question says "with appropriate example" and "assume and mention the data". Always write a line saying "Assumed data:" and show a small table. Examples are where marks are won.

Master Plan of the Four Units

UNIT 1	UNIT 2	UNIT 3	UNIT 4
Intro, Data Models & Architecture	Relational Model & Keys	SQL	Normalization
-----	-----	-----	-----
DBMS & its goal	Tables, rows, cols	CREATE TABLE *	Why normalize
File-processing	Null values	ALTER TABLE *	Anomalies
3 abstraction levels	KEYS (all types) *	Set operations *	Functional dep.
Data models	Artificial key	JOINS *	1NF
DDL / DML	Referential integrity *	HAVING, EXISTS	2NF
Architecture	Other constraints	Subqueries	3NF
Users + DBA *		INSERT/UPDATE	BCNF
ER model			(* = top priority)

Contents

💧 high-yield ★ essential 💡 possible surprise 📊 foundation

Unit 1: Introduction, Data Models and Architecture

- 1.1 What is a DBMS and What is its Primary Goal? ★
- 1.2 Where are Databases Used? 📊
- 1.3 File-Processing System: Pros and Cons ★
- 1.4 View of Data: Three Levels of Abstraction 💡
- 1.5 Schema and Instance 📊
- 1.6 Data Models and Their Categories ★
- 1.7 Database Languages: DDL and DML 💡
- 1.8 Database Architecture and Components of a Database System ★
- 1.9 Database Users and the Database Administrator (DBA) 🔥
- 1.10 The Entity-Relationship (E-R) Model and Mapping Cardinality ★

Unit 2: Relational Model and Keys

- 2.1 The Relational Model: Basic Terms 📊
- 2.2 Null Values 📊
- 2.3 Keys in a Database 🔥
- 2.4 Integrity Constraints and Referential Integrity 🔥

Unit 3: SQL (Structured Query Language)

- 3.1 Parts of SQL 💡
- 3.2 Basic Data Types 📊
- 3.3 CREATE TABLE 🔥
- 3.4 ALTER TABLE and DROP TABLE 🔥
- 3.5 Basic Query Structure: SELECT, FROM, WHERE 📊
- 3.6 Set Operations: UNION, INTERSECT, EXCEPT 🔥
- 3.7 Joins 🔥
- 3.8 Aggregate Functions, GROUP BY and HAVING ★
- 3.9 Nested Subqueries: IN, EXISTS, SOME, ALL ★
- 3.10 Modifying Data: INSERT, UPDATE, DELETE 💡

Unit 4: Normalization

- 4.1 What is Normalization and Why is it Important? 🔥
- 4.2 Functional Dependency: The Idea Behind Every Normal Form 📊
- 4.3 First Normal Form (1NF) 🔥
- 4.4 Second Normal Form (2NF) 🔥
- 4.5 Third Normal Form (3NF) 🔥
- 4.6 One Complete Example: UNF to 1NF to 2NF to 3NF 🔥
- 4.7 Boyce-Codd Normal Form (BCNF) 💡
- 4.8 Normal Forms at a Glance 🔥

Quick Revision Section

- A. Past-Paper Question Bank with Answer Locations
- B. Answer-Writing Templates
- C. One-Page Cheat Sheet
- D. Last-Minute Checklist

Unit 1: Introduction, Data Models and Architecture

1.1 What is a DBMS and What is its Primary Goal? [★ ESSENTIAL FOR EXAM]

Data means raw facts (a name, a number). A **database** is an organised collection of related data. A **Database Management System (DBMS)** is the software that lets you store, retrieve, update and protect that data.

Analogy: A database is a huge library. The DBMS is the librarian. You never walk into the storage room and search shelves yourself; you ask the librarian, who finds, issues, returns and protects the books for you.

Definition (write this in the exam): A DBMS is a collection of interrelated data and a set of programs to access that data.

Primary goal of a DBMS: to store and retrieve database information in a way that is both **convenient** and **efficient**.

What a DBMS must do:

- **Define structures** for storing information (tables, columns, data types).
- **Manipulate information** (insert, update, delete, retrieve).
- **Keep data safe** despite system crashes or unauthorised access.
- **Avoid anomalous results** when many users share the same data.

Example: A university database stores students, courses and grades. The registrar asks "Which students enrolled in CS-101?". The DBMS answers in seconds without anyone opening files by hand.

```
User / Program
|
| "Give me all students in CS-101"
v
+-----+
|      DBMS      | <- the "librarian"
| (finds, protects, |
| updates, shares) |
+-----+
|
| v
+-----+
|      DATABASE   | <- the "library"
| Student | Course |
| Grades  | Fees ... |
+-----+
```

1.2 Where are Databases Used? [■ FOUNDATION: Background Concept]

Area	What the database stores
Banking and Finance	Customers, accounts, loans, credit-card transactions
Universities	Student records, course registration, grades
Airlines	Reservations and schedules
Telecommunication	Call records, monthly bills, prepaid balances
Online retail	Orders, product reviews, recommendations
Enterprise (HR, Sales, Accounting)	Employees, payroll, sales, payments

1.3 File-Processing System: Pros and Cons [★ ESSENTIAL FOR EXAM]

Before DBMSs, organisations stored data in ordinary **operating-system files** (text files, spreadsheets) and wrote a separate program for every task. This is the **file-processing system**.

Analogy: Imagine every department of a college keeping its own paper register. The fees office has its list of students, the exam office has another, and the hostel has a third. The same student's address appears in three places, and when a student moves house, someone must fix all three.

Advantages (pros)

- **Simple and cheap** for a very small, single-purpose task.
- **Quick to start:** no special software is needed beyond the operating system.
- **Fast for one fixed job**, because the program is written exactly for that file.

Disadvantages (cons): the seven problems a DBMS solves

#	Problem	Meaning	Example
1	Data redundancy and inconsistency	The same data is stored in many files, and copies disagree	A student's phone number differs in the fees file and the exam file
2	Difficulty in accessing data	A new question needs a new program	Clerk wants "all students in postal code 380009"; no program exists, so a programmer must write one
3	Data isolation	Data is scattered across files in different formats	One file is CSV, another is a text file
4	Integrity problems	Rules (constraints) are buried inside programs	"Balance must never be below 0" must be re-coded in every program
5	Atomicity problems	A failure can leave a task half-done	Rs 500 deducted from account A, then the system crashes before it is added to B
6	Concurrent-access anomalies	Two users updating together overwrite each other	Two clerks update the same balance at the same moment
7	Security problems	Hard to restrict who sees what	Payroll staff should see salaries but not exam marks

Memory trick: R-A-D-I-A-C-S = Redundancy, Access, Data isolation, Integrity, Atomicity, Concurrent access, Security.

FILE-PROCESSING SYSTEM	DBMS APPROACH
-----	-----
Fees App ---> fees_file.txt	Fees App ---+
Exam App ---> exam_file.csv	Exam App ---+--> [DBMS] --> ONE shared
Hostel App -> hostel_file.dat	Hostel App -+ database
(3 copies of student address)	(1 copy, everyone sees it)

Exam Tip: For "pros and cons", write 2-3 pros, then the 7 cons in a table with one example each. This reaches full marks for a 6-mark question.

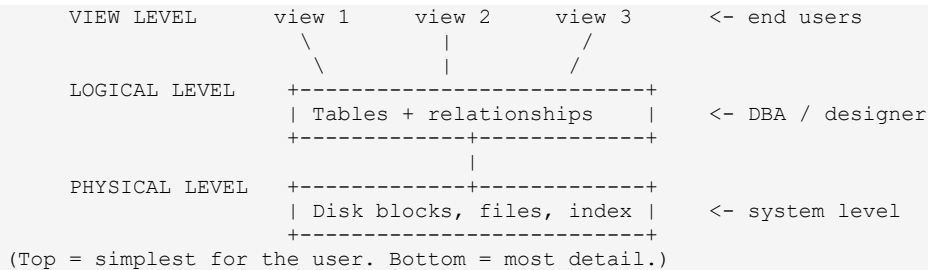
1.4 View of Data: Three Levels of Abstraction [💡 POSSIBLE SURPRISE: In Syllabus, Not Yet Asked]

Data abstraction means hiding complex details and showing users only what they need.

Analogy: Driving a car. As a driver (view level) you only see the steering wheel and pedals. A mechanic (logical level) knows the parts and how they connect. An engineer (physical level) knows the metal, the wiring and how the engine burns fuel.

Level	Describes	Who uses it	Example
Physical level (lowest)	HOW data is actually stored (files, disk blocks, data structures)	System developers	Tables stored as blocks on hard disks in a data centre

Level	Describes	Who uses it	Example
Logical level (middle)	WHAT data is stored and the relationships among them	DBAs, designers	Tables <code>product</code> and <code>customer</code> , and a join showing who ordered what
View level (highest)	Only PART of the database for a particular user	End users	A customer sees only their own order history; a shop owner sees the product order list



Data independence: a change at a lower level does not force a change at the higher level.

- **Physical data independence:** you can change how data is stored (e.g., move to a faster disk) and the logical tables stay the same.

1.5 Schema and Instance [FOUNDATION: Background Concept]

- **Schema** = the **design** (structure) of the database. It rarely changes.
- **Instance** = the **actual data** in the database at one moment. It changes constantly.

Analogy: The schema is an empty exam answer-sheet template (columns and headings). The instance is the same sheet after students have filled in their answers.

Example:

```

SCHEMA (design):      student(roll_no, name, city)

INSTANCE (data at 10 AM):      INSTANCE (data at 4 PM):
roll_no | name | city      roll_no | name | city
-----+-----+-----      -----+-----+-----
1       | Asha | Surat      1       | Asha | Surat
2       | Ravi | Vadodara    2       | Ravi | Vadodara
3       | Meena | Rajkot      3       | Meena | Rajkot <- new row

```

1.6 Data Models and Their Categories [★ ESSENTIAL FOR EXAM]

A **data model** is a collection of conceptual tools for describing **data**, **data relationships**, **data semantics** and **constraints**. It is the "language" used to describe how data is organised.

Analogy: To describe a city you can use a street map, a list of addresses, or a family tree of who lives with whom. Each is a different "model" of the same city. Likewise a database can be modelled as tables, as entities and relationships, or as objects.

The four categories in your syllabus

Data model	Core idea	Example
Relational model	Data in tables (rows and columns); tables connected through common columns	<code>student</code> table + <code>enrollment</code> table linked by <code>student_id</code>
Entity-Relationship (E-R) model	Real world as entities and relationships between them	Entities Instructor and Department; relationship "works in"

Data model	Core idea	Example
Object-based model (object-oriented / object-relational)	Data stored as objects with data AND methods (functions)	A Car object with data (colour, speed) and methods (start, stop)
Semi-structured model	Structure is flexible ; each record can have different fields (XML, JSON)	JSON documents where one person has a phone number and another does not

RELATIONAL	E-R MODEL	OBJECT-BASED
<pre> +-----+ ID Name +-----+ 1 Asha 2 Ravi +-----+ </pre>	<pre> [Instructor]--<works>--[Department] (entity) (relationship) (entity) </pre>	<pre> +-----+ Car colour speed start() +-----+ </pre>
	<pre> SEMI-STRUCTURED (JSON) { "name": "Asha", "phone": "98..." } { "name": "Ravi" } <- no phone: allowed </pre>	

Key characteristics of the semi-structured model: flexible schema, self-describing (data carries its own structure), irregular structure, easier integration of different sources, human-readable text formats.

Another way to classify (useful for a 6-mark answer):

- **Conceptual (high-level) models:** close to how humans think (E-R model).
- **Representational (implementation) models:** used by DBMS software (relational model).
- **Physical (low-level) models:** describe how data is stored on disk.

1.7 Database Languages: DDL and DML [💡 POSSIBLE SURPRISE: In Syllabus, Not Yet Asked]

	DDL (Data-Definition Language)	DML (Data-Manipulation Language)
Purpose	Defines the structure (schema)	Works with the data inside the structure
Commands	CREATE, ALTER, DROP	SELECT, INSERT, UPDATE, DELETE
Analogy	Building the empty shelves	Putting books on, taking books off the shelves
Output	Stored in the data dictionary	Changes the stored data

```

-- DDL: builds the structure
CREATE TABLE student (roll_no NUMBER(5), name VARCHAR2(30));

-- DML: works with the data
INSERT INTO student VALUES (1, 'Asha');
SELECT name FROM student WHERE roll_no = 1;

```

Two types of DML:

- **Procedural DML:** you say **what** data you need AND **how** to get it (e.g., PL/SQL).
- **Declarative (non-procedural) DML:** you say only **what** you need (e.g., SQL). The DBMS decides how.

Analogy: Declarative = telling a taxi "take me to the airport". Procedural = giving the driver every turn: "left, then right, then straight".

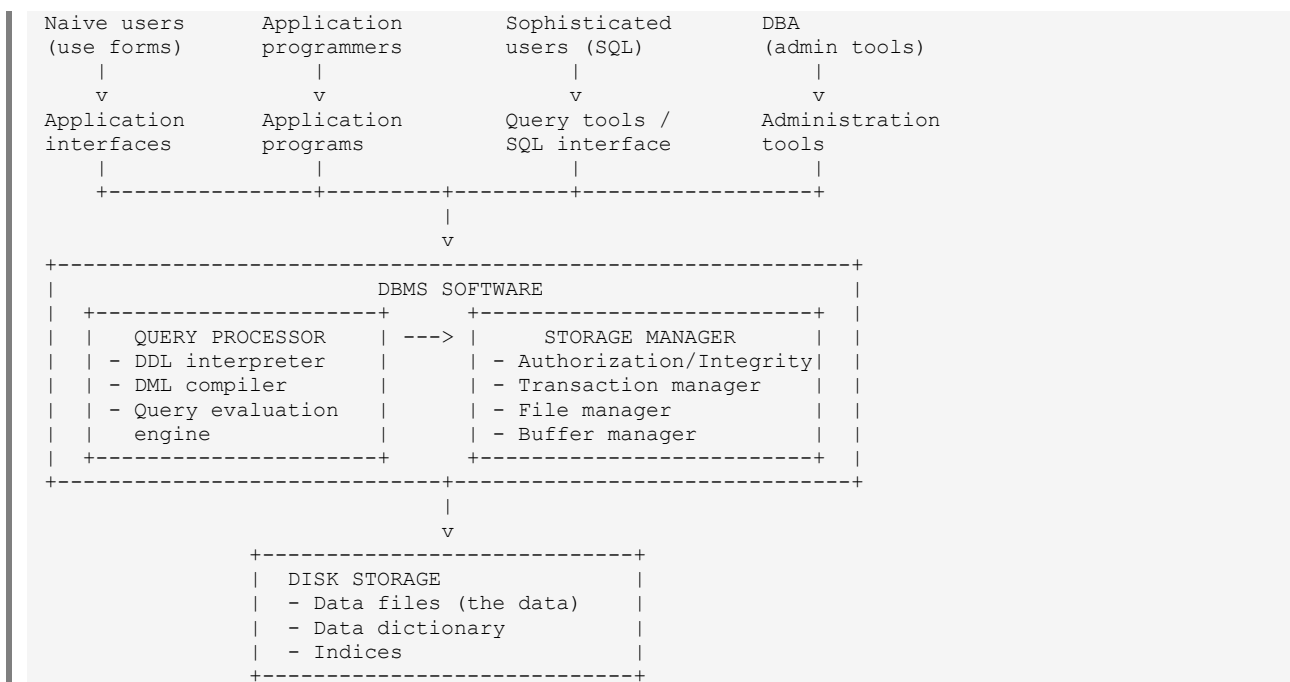
Data dictionary (metadata): a special table-set holding the schema, data types, constraints and indices, i.e., "data about data".

1.8 Database Architecture and Components of a Database System [★ ESSENTIAL FOR EXAM]

The Feb-22 paper asked: "Discuss the various components of a database system and the connections among them with an appropriate diagram." Learn **both diagrams** below.

A) Overall components of a database system

Component	Role
Users	Naive users, application programmers, sophisticated users, specialized users, DBA
Application programs / interfaces	Forms, reports and programs through which users reach the database
DBMS software	Query processor + storage manager
Database	The stored data, the data dictionary and the indices
DBA	Person with central control of the whole system



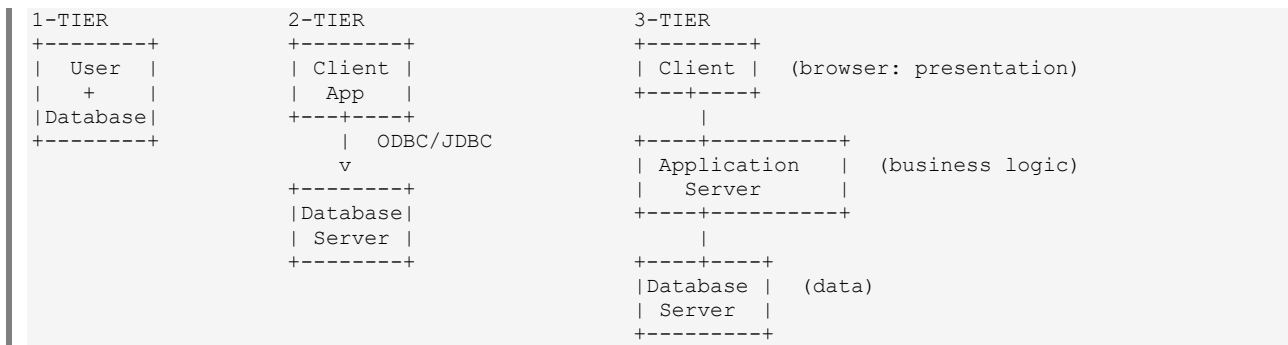
Query processor parts (from your syllabus):

- **DDL interpreter:** reads DDL statements and records the definitions in the data dictionary.
- **DML compiler:** translates DML/SQL into low-level instructions and picks the best evaluation plan (query optimisation).
- **Query evaluation engine:** executes those low-level instructions.

Storage manager (standard textbook components): the interface between stored data and the queries. It includes the authorization and integrity manager, transaction manager, file manager and buffer manager.

B) 1-tier, 2-tier and 3-tier architecture

Architecture	How it works	Example
1-Tier	The user sits directly on the DBMS. Changes happen directly in the database.	A programmer testing queries on a local Oracle installation
2-Tier	Client application talks directly to the database server using APIs like ODBC / JDBC .	A desktop billing program connecting to a shop's database server
3-Tier	An application server sits between client and database. The client never touches the database directly.	An online shopping website: browser, web/app server, database



Why 3-tier? Better **security** (client cannot reach the database), better **scalability**, and business rules are kept in one place.

1.9 Database Users and the Database Administrator (DBA) [🔥 HIGH-YIELD: Frequently Asked]

Types of database users

User type	Who they are	How they use the DB	Example
Naive users	Unsophisticated end users	Run pre-written application programs	Bank teller using a "transfer" screen
Application programmers	Computer professionals	Write programs using APIs like ODBC/JDBC	Developer building a banking app
Sophisticated users	Know database concepts	Write queries directly in SQL	Data analyst: <code>SELECT department, AVG(salary) FROM employees GROUP BY department;</code>
Specialized users	Advanced users	Build special applications that do not fit normal data-processing	Medical researcher storing images and genetic sequences

Database Administrator (DBA): roles and functions

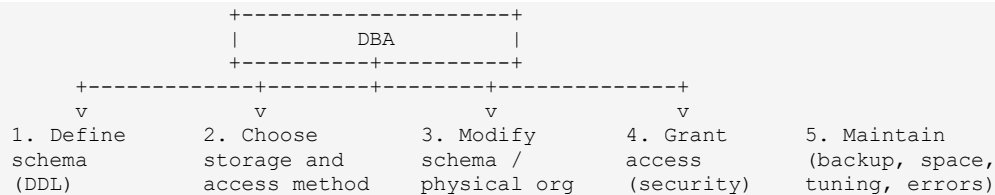
The **DBA** is the person with **central control** over the data and the programs that access it. The Jan-23 (9 marks) and Jan-25 (8 marks) papers both asked this.

Analogy: The DBA is the **chief librarian and security head** of the library in one person. They design the shelving system, decide who may enter which section, repair damage, take backups and keep everything running.

#	Function	What the DBA does	Example
1	Schema definition	Creates the original database schema using DDL statements	Runs <code>CREATE TABLE</code> for all tables
2	Storage structure and access-method definition	Chooses suitable storage structures and access methods	Decides to create an index on <code>student_id</code>
3	Schema and physical-organisation modification	Changes the schema or physical layout as needs change or to improve performance	Adds a new column; moves data to a faster disk
4	Granting authorisation for data access	Controls which users can access which part of the data	<code>GRANT SELECT ON salary TO payroll_user;</code>
5	Routine maintenance	Keeps the system healthy (see below)	Nightly backup at 2 AM

Routine maintenance includes:

- **Backups:** periodically back up to tapes or remote servers to prevent loss from disasters like floods.
- **Disk space:** ensure enough free disk space and upgrade when needed.
- **Monitoring jobs:** make sure expensive queries do not slow down the system.
- **Database tuning:** maximise resources and improve efficiency.
- **Error checking:** check for database and server errors.



Exam Tip: Write the five functions as numbered headings, with 1-2 lines and one example each. Draw the small DBA diagram. This format is enough for 8-9 marks.

1.10 The Entity-Relationship (E-R) Model and Mapping Cardinality [★ ESSENTIAL FOR EXAM]

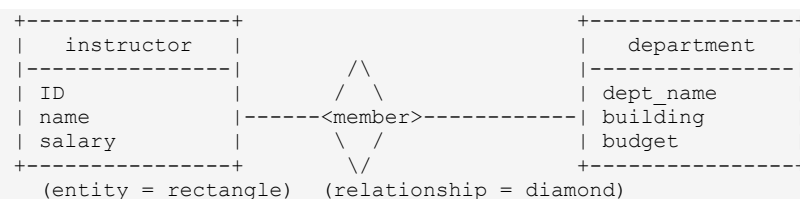
The **E-R model** describes a database as a set of **entities** and the **relationships** among them. It was asked in Jan-23 with cardinality (7 marks). Your PDF introduces it briefly, so also check your class notes.

Analogy: An E-R diagram is a **family-tree style sketch** of the college: boxes for the people and things (students, courses), and diamonds that say how they are connected ("enrolls in").

Basic terms:

- **Entity:** a "thing" in the real world that can be told apart from others (a student, a course).
- **Entity set:** all entities of the same type (all students).
- **Attribute:** a property of an entity (name, roll number).
- **Relationship:** an association among entities (a student **enrolls in** a course).
- **Relationship set:** all relationships of the same type.

Example diagram (instructor and department):



Mapping cardinality

Mapping cardinality = the **number of entities** to which another entity can be associated via a relationship set.

Type	Meaning	Real-world example
One-to-One (1:1)	One A is linked to at most one B, and one B to at most one A	One person has one passport
One-to-Many (1:N)	One A can be linked to many B; each B to only one A	One department has many instructors
Many-to-One (N:1)	Many A link to one B	Many instructors belong to one department

Type	Meaning	Real-world example
Many-to-Many (M:N)	Many A link to many B	Many students enrol in many courses

ONE-TO-ONE	ONE-TO-MANY	MANY-TO-ONE	MANY-TO-MANY
A B	A B	A B	A B
a1 --- b1	a1 --- b1	a1 --- b1	a1 --- b1
a2 --- b2	a1 --- b2	a2 --- b1	a1 --- b2
a3 --- b3	a2 --- b3	a3 --- b2	a2 --- b2
			a3 --- b3
(each A has one B)	(one A has many B)	(many A share one B)	(links go both ways in plenty)

Exam Tip: Draw the E-R diagram of one example (e.g., Student-Course), label the cardinality (1, N, M), and give one real-life example for each of the four types.

Unit 2: Relational Model and Keys

2.1 The Relational Model: Basic Terms [FOUNDATION: Background Concept]

The **relational model** is today the main data model for commercial applications because of its **simplicity**. A relational database is a collection of **tables** (also called **relations**), each with a unique name.

Analogy: A table is like a **register or Excel sheet**: the headings are the columns, and every line below is one record.

Formal term	Everyday term	Meaning
Relation	Table	A named collection of rows and columns
Tuple	Row / record	One entry in the table
Attribute	Column / field	One property of the entity
Domain	Allowed values	The set of legal values for an attribute (e.g., salary must be a positive number)
Degree	Number of columns	A table with 4 columns has degree 4
Cardinality	Number of rows	A table with 100 rows has cardinality 100

Example: the `instructor` relation

```

attributes (columns)
+-----+-----+-----+-----+
| ID | name | dept_name | salary | <- schema (headings)
+-----+-----+-----+-----+
tuple-> | 10101 | Srinivas | Comp. Sci. | 65000 |
(row)   | 12121 | Wu       | Finance    | 90000 |
        | 22222 | Einstein | Physics    | 95000 |
+-----+-----+-----+-----+
Degree = 4 columns      Cardinality = 3 rows

```

Properties of a relation:

- Each table has a **unique name**, and each column has a unique name within the table.
- Values in a column are from the **same domain** (all salaries are numbers).
- **Row order does not matter** and there are **no duplicate rows**.
- Values are **atomic** (one value per cell), which links to First Normal Form in Unit 4.

Tables also store relationships. In the example below the `student_id` column connects the two tables:

STUDENT			ENROLLMENT	
student_id	name		student_id	course_id
1	Asha	<-----	1	CS-101
2	Ravi	<-----	2	CS-101

2.2 Null Values [FOUNDATION: Background Concept]

A **null** is a special value meaning **unknown or does not exist**. Null is **not zero** and **not an empty string**.

Analogy: A blank on a form. "Phone: ____" could mean the person has no phone, or that you have not asked yet. Zero would claim "the phone number is 0", which is wrong.

Example: An instructor who has just joined and has no salary assigned yet has `salary = NULL`.

Rules to remember:

- Test for null with `IS NULL` / `IS NOT NULL`. Never write `= NULL`.
- Comparison with null gives **unknown**, so `salary > 50000` does not select a null salary.
- Aggregate functions (`SUM`, `AVG`, `MIN`, `MAX`, `COUNT(column)`) **ignore nulls**; only `COUNT(*)` counts every row.
- A **primary key can never be null**.

```
SELECT name FROM instructor WHERE salary IS NULL;
```

2.3 Keys in a Database [🔥 HIGH-YIELD: Frequently Asked]

Keys were asked in **all four** past papers. The question usually starts "Why do we need a SQL key...?" and then lists several key types.

Why do we need keys?

A **key** is an attribute (or a group of attributes) that helps us identify a row in a table.

- **Unique identification:** find one exact row among thousands (one roll number = one student).
- **Avoid duplicates:** two rows cannot have the same key value.
- **Link tables:** keys connect one table to another (foreign key).
- **Fast searching:** keys can be indexed so the DBMS finds rows quickly.
- **Enforce integrity:** keys protect the correctness of data.

Analogy: A key is like an **Aadhaar number or roll number**. Names can repeat (two students called "Ravi"), but the roll number points to exactly one person.

One table to understand all keys

Use this `STUDENT` table for every definition below.

STUDENT					
STU_ID	ENROLL_NO	NAME	EMAIL	PHONE	
1	2024MCA01	Asha Shah	asha@nirma.ac.in	9876500001	
2	2024MCA02	Ravi Patel	ravi@nirma.ac.in	9876500002	
3	2024MCA03	Ravi Patel	ravi2@nirma.ac.in	9876500003	

(Two students share the name "Ravi Patel", so NAME alone cannot be a key.)

1. Super Key

Definition: A set of one or more attributes that **uniquely identifies** a row. It may contain extra, unnecessary attributes.

Example: `{STU_ID}`, `{ENROLL_NO}`, `{EMAIL}`, `{STU_ID, NAME}`, `{STU_ID, NAME, PHONE}`, `{ENROLL_NO, EMAIL}`... All of these are super keys because each combination identifies a row uniquely.

`{NAME}` is **not** a super key (two Ravi Patels).

2. Candidate Key

Definition: A **minimal** super key: a super key from which **no attribute can be removed** without losing uniqueness.

Example: {STU_ID}, {ENROLL_NO}, {EMAIL} and {PHONE} are candidate keys. {STU_ID, NAME} is a super key but **not** a candidate key, because removing NAME still leaves a unique STU_ID.

3. Primary Key

Definition: The **one candidate key** chosen by the designer to identify rows. It must be **unique and NOT NULL**, and a table has **only one**.

Example: The designer picks STU_ID as the primary key.

```
CREATE TABLE student (
    stu_id    NUMBER(5) PRIMARY KEY,
    enroll_no VARCHAR2(12) UNIQUE,
    name      VARCHAR2(30),
    email     VARCHAR2(40) UNIQUE
);
```

4. Alternate Key (Secondary Key)

Definition: Every candidate key that was **not chosen** as the primary key.

Example: With STU_ID as primary key, the alternate keys are ENROLL_NO, EMAIL and PHONE.

5. Foreign Key

Definition: An attribute in one table that **refers to the primary key of another table**. It creates the link between two tables.

Example:

DEPARTMENT (parent)			INSTRUCTOR (child)		
DEPT_NAME	BUILDING		ID	NAME	DEPT_NAME
CompSci	Taylor		10101	Srini	CompSci
Physics	Watson		22222	Einst	Physics

^ primary key <- foreign key

A foreign key value may be **repeated** and may be **null** (unless declared NOT NULL), but a non-null value must exist in the parent table.

6. Composite Key

Definition: A key made of **two or more attributes together**, used when **no single column** is unique by itself.

Example: A SCORE table stores marks of each student in each subject. A student appears many times (once per subject) and a subject appears many times (once per student), so neither column alone is unique. Together they are.

STUDENT_ID	SUBJECT_ID	MARKS
1	DBMS	78
1	JAVA	82
2	DBMS	65

Composite primary key = (STUDENT_ID, SUBJECT_ID)

```
CREATE TABLE score (
    student_id NUMBER(5),
    subject_id VARCHAR2(10),
    marks      NUMBER(3),
    PRIMARY KEY (student_id, subject_id)
);
```

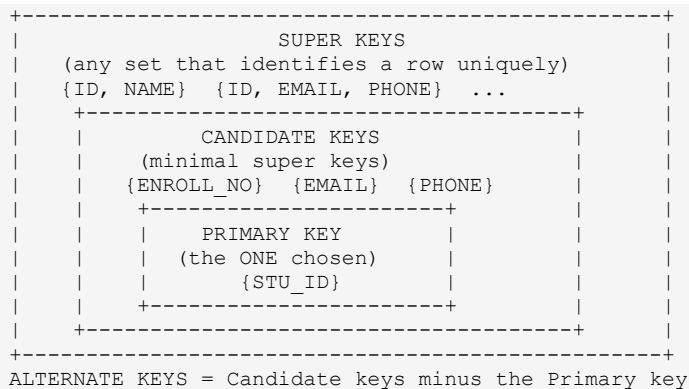
7. Artificial Key (Surrogate Key) [★ ESSENTIAL FOR EXAM]

Definition: A key created with **arbitrarily assigned values** (usually an auto-number) because the natural key is **large, complex, or has no meaning** to other tables. Your syllabus defines it as a key created from arbitrarily assigned data when the primary key is large and complex.

Example: To identify a flight booking you could use `(passenger_name, flight_no, date, seat_no)`, which is long and clumsy. Instead, the system creates `BOOKING_ID = 100245`, an artificial key.

```
CREATE SEQUENCE booking_seq START WITH 1; -- generates 1, 2, 3 ...
INSERT INTO booking (booking_id, passenger_name, flight_no)
VALUES (booking_seq.NEXTVAL, 'Asha Shah', 'AI-101');
```

How the keys relate to one another



Remember: Every **primary key** is a **candidate key**, and every **candidate key** is a **super key**. But not every super key is a candidate key.

Quick comparison table

Key	Unique?	Null allowed?	How many per table?	One-line meaning
Super key	Yes	Depends	Many	Any set that identifies a row
Candidate key	Yes	Depends	Many	Minimal super key
Primary key	Yes	No	One	Chosen candidate key
Alternate key	Yes	Depends	Many	Candidate keys not chosen as primary
Foreign key	No (can repeat)	Yes	Many	Refers to another table's primary key
Composite key	Yes (together)	No	Any	Key made of 2+ columns
Artificial key	Yes	No	Any	System-generated key with no real-world meaning

Exam Tip: Start with "Why do we need keys?" (3-4 lines), then define each key with the STUDENT table, show the nested-ellipse diagram, and end with the comparison table. This covers every version of this question.

2.4 Integrity Constraints and Referential Integrity [🔥 HIGH-YIELD: Frequently Asked]

Integrity constraints are rules that guard the **accuracy and consistency** of data, so that changes by authorised users cannot damage it.

Referential integrity

Definition: Referential integrity ensures that a value that appears in one relation (the **child**) for a given set of attributes also appears in another relation (the **parent**). In plain words: **a foreign key value must either be NULL or match an existing primary key value in the parent table.**

Analogy: A library issue card may only mention a book that actually exists in the library catalogue. You cannot issue a "ghost book" that is not in the catalogue. Likewise a student cannot be enrolled in a course that does not exist.

Example: Tables `department` (parent) and `instructor` (child).

DEPARTMENT (parent)			INSTRUCTOR (child)			
DEPT_NAME	BUILDING		ID	NAME	DEPT_NAME	
CompSci	Taylor		10101	Srini	CompSci	OK
Physics	Watson		22222	Einst	Physics	OK
			33333	Gupta	Maths	REJECTED

'Maths' does not exist in DEPARTMENT, so the insert fails.

```
CREATE TABLE department (
  dept_name VARCHAR2(20) PRIMARY KEY,
  building VARCHAR2(15)
);
CREATE TABLE instructor (
  id VARCHAR2(5) PRIMARY KEY,
  name VARCHAR2(20),
  dept_name VARCHAR2(20),
  FOREIGN KEY (dept_name) REFERENCES department(dept_name)
);
INSERT INTO instructor VALUES ('33333', 'Gupta', 'Maths');
-- ERROR: integrity constraint violated - parent key not found
```

What happens on delete or update in the parent? You choose the behaviour:

Option	Effect
(default)	The delete/update is rejected if child rows exist
<code>ON DELETE CASCADE</code>	Child rows are deleted automatically
<code>ON DELETE SET NULL</code>	The child's foreign key is set to NULL

```
FOREIGN KEY (dept_name) REFERENCES department(dept_name) ON DELETE SET NULL
```

Other integrity constraints [💡 POSSIBLE SURPRISE: In Syllabus, Not Yet Asked]

Constraint	What it enforces	Example
Domain constraint	Values in a column must belong to the allowed domain / type	<code>salary NUMBER(8,2) CHECK (salary > 0)</code>
NOT NULL	Column cannot be empty	<code>name VARCHAR2(20) NOT NULL</code>
UNIQUE	No duplicates in a column (nulls allowed)	<code>email VARCHAR2(40) UNIQUE</code>
PRIMARY KEY	Unique + NOT NULL; identifies rows	<code>PRIMARY KEY (stu_id)</code>
FOREIGN KEY	Referential integrity between tables	<code>REFERENCES department(dept_name)</code>
CHECK	A custom condition on values	<code>CHECK (semester IN ('Fall','Spring'))</code>

Constraint	What it enforces	Example
Assertion	Any condition the whole database must always satisfy	"Total credits of a student must equal the sum of credits of passed courses"

Unit 3: SQL (Structured Query Language)

Sample data used in this unit. Keep these two tables in mind; most examples use them.

INSTRUCTOR			
ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000

3.1 Parts of SQL [🔗 POSSIBLE SURPRISE: In Syllabus, Not Yet Asked]

SQL is **non-procedural** (declarative): you say **what** you want, not **how** to get it. A query takes one or more tables as input and **always returns a single table**.

Part	Full form	Job	Commands
DDL	Data Definition Language	Create / change / delete table structure	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Data Manipulation Language	Query and change data	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	Access rights (authorisation)	GRANT, REVOKE
TCL	Transaction Control Language	Begin / end / undo transactions	COMMIT, ROLLBACK, SAVEPOINT

Analogy: DDL builds the house, DML moves the furniture in and out, DCL decides who gets keys, and TCL is the "save / undo" button.

3.2 Basic Data Types [📌 FOUNDATION: Background Concept]

Type (Oracle)	Meaning	Example
CHAR(n)	Fixed-length text of n characters (padded with spaces)	CHAR(2) for state code 'GJ'
VARCHAR2(n) (varchar(n))	Variable-length text, maximum n characters	VARCHAR2(30) for a name
NUMBER(p,d) (numeric(p,d))	Number with p total digits, d after the decimal point	NUMBER(12,2) stores 1234567890.50
INT / NUMBER(n)	Whole number	credits = 4
DATE	Date (and time in Oracle)	'15-JAN-2026'

Reading `NUMBER(8,2)`: 8 digits in total, 2 of them after the decimal point, so the largest value is 999999.99.

3.3 CREATE TABLE [🔥 HIGH-YIELD: Frequently Asked]

This is the **most repeated high-mark question**: Jan-22, Feb-22 and Jan-23 all asked it with the **same university schema**, 9 marks each.

Question as asked: "Write SQL for creating a table corresponding to the schema below. Make any reasonable assumptions about data types, primary key and reference key."

Basic syntax

```
CREATE TABLE table_name (
    column1 datatype [column constraint],
    column2 datatype [column constraint],
    ...
    [table constraints: PRIMARY KEY (...), FOREIGN KEY (...) REFERENCES ...]
);
```

Mini example:

```
CREATE TABLE student (
    roll_no NUMBER(5) PRIMARY KEY,
    name VARCHAR2(30) NOT NULL,
    city VARCHAR2(20),
    fees NUMBER(8,2) CHECK (fees >= 0)
);
```

The university schema

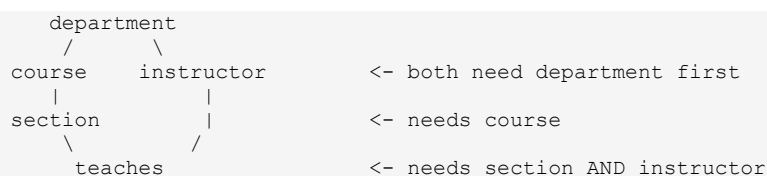
```
department(dept_name, building, budget)
course(course_id, title, dept_name, credits)
instructor(ID, name, dept_name, salary)
section(course_id, sec_id, semester, year, building, room_number, time_slot_id)
teaches(ID, course_id, sec_id, semester, year)
```

Step 1: Decide keys BEFORE writing code

Table	Primary key	Foreign keys	Reason
department	dept_name	none	Each department name is unique
course	course_id	dept_name references department	A course belongs to a department
instructor	ID	dept_name references department	An instructor belongs to a department
section	(course_id, sec_id, semester, year) composite	course_id references course	One course has many sections in different semesters and years
teaches	(ID, course_id, sec_id, semester, year) composite	ID references instructor; (course_id, sec_id, semester, year) references section	Links an instructor to the sections they teach

Step 2: Create tables in the right ORDER

A table that is **referred to** must exist **before** the table that refers to it.



```
CREATE ORDER: department -> course -> instructor -> section -> teaches
DROP ORDER : the reverse (teaches first, department last)
```

Step 3: The complete script (write this in the exam)

```
-- Assumptions: standard university schema; the names/sizes below are my choice.

CREATE TABLE department (
    dept_name VARCHAR2(20),
    building   VARCHAR2(15),
    budget     NUMBER(12,2) CHECK (budget > 0),
    PRIMARY KEY (dept_name)
);

CREATE TABLE course (
    course_id VARCHAR2(8),
    title      VARCHAR2(50),
    dept_name  VARCHAR2(20),
    credits    NUMBER(2,0) CHECK (credits > 0),
    PRIMARY KEY (course_id),
    FOREIGN KEY (dept_name) REFERENCES department(dept_name)
        ON DELETE SET NULL
);

CREATE TABLE instructor (
    ID          VARCHAR2(5),
    name        VARCHAR2(20) NOT NULL,
    dept_name   VARCHAR2(20),
    salary      NUMBER(8,2) CHECK (salary > 29000),
    PRIMARY KEY (ID),
    FOREIGN KEY (dept_name) REFERENCES department(dept_name)
        ON DELETE SET NULL
);

CREATE TABLE section (
    course_id   VARCHAR2(8),
    sec_id      VARCHAR2(8),
    semester    VARCHAR2(6) CHECK (semester IN ('Fall','Winter','Spring','Summer')),
    year        NUMBER(4,0) CHECK (year > 1701 AND year < 2100),
    building    VARCHAR2(15),
    room_number VARCHAR2(7),
    time_slot_id VARCHAR2(4),
    PRIMARY KEY (course_id, sec_id, semester, year),
    FOREIGN KEY (course_id) REFERENCES course(course_id)
        ON DELETE CASCADE
);

CREATE TABLE teaches (
    ID          VARCHAR2(5),
    course_id   VARCHAR2(8),
    sec_id      VARCHAR2(8),
    semester    VARCHAR2(6),
    year        NUMBER(4,0),
    PRIMARY KEY (ID, course_id, sec_id, semester, year),
    FOREIGN KEY (course_id, sec_id, semester, year)
        REFERENCES section(course_id, sec_id, semester, year) ON DELETE CASCADE,
    FOREIGN KEY (ID) REFERENCES instructor(ID) ON DELETE CASCADE
);
```

Why each choice was made (mention these lines in the exam):

- **'VARCHAR2'** for text, **'NUMBER(12,2)'** for money (2 decimals), **'NUMBER(2,0)'** for small whole numbers.
- **'NOT NULL'** on `instructor.name`: an instructor must have a name.
- **'CHECK'** constraints protect the domain (budget and salary must be sensible).
- **Composite primary key** in `section` and `teaches` because no single column is unique.
- **'ON DELETE CASCADE'**: if a course is deleted, its sections go too. **'ON DELETE SET NULL'**: if a department is deleted, courses and instructors keep existing with no department.

Exam Tip: Always write (1) a one-line assumption, (2) tables in the correct order, (3) primary key and foreign key for every table. Missing a composite key in `section` or `teaches` is the most common lost mark.

3.4 ALTER TABLE and DROP TABLE [🔥 HIGH-YIELD: Frequently Asked]

ALTER TABLE changes the structure of a table that **already exists**, without deleting it. Jan-25 asked five operations for 8 marks.

Analogy: You already built a bookshelf (the table). **ALTER** is adding a new shelf, removing one, relabelling one or making a shelf bigger, without throwing away the books.

Use this table for all examples:

```
INSTRUCTOR (before any change)
+-----+-----+-----+-----+
| ID | name | dept_name | salary |
+-----+-----+-----+-----+
| 10101 | Srini | Comp. Sci. | 65000 |
| 22222 | Einstein | Physics | 95000 |
+-----+-----+-----+-----+
```

I. Add a new attribute (column)

```
ALTER TABLE instructor ADD email VARCHAR2(40);
```

Result: a new column **email**; every existing row gets **NULL** in it.

```
+-----+-----+-----+-----+-----+
| ID | name | dept_name | salary | email |
+-----+-----+-----+-----+-----+
| 10101 | Srini | Comp. Sci. | 65000 | NULL |
| 22222 | Einstein | Physics | 95000 | NULL |
+-----+-----+-----+-----+-----+
```

Add several columns at once (Oracle): `ALTER TABLE instructor ADD (phone VARCHAR2(10), joined DATE);`

II. Drop an attribute from an existing relation

```
ALTER TABLE instructor DROP COLUMN email;
```

The column and all its data are **permanently removed**. (Your textbook notes that some databases do not support dropping attributes, but Oracle, MySQL and PostgreSQL do.)

III. Rename a column

```
ALTER TABLE instructor RENAME COLUMN name TO instructor_name;
```

IV. Modify the data type (or size) of a column

```
ALTER TABLE instructor MODIFY salary NUMBER(10,2);
```

Here the salary column grows from **NUMBER(8,2)** to **NUMBER(10,2)**. **Careful:** shrinking a column below the size of existing data, or changing to an incompatible type, gives an error.

V. Add a NOT NULL constraint to an existing column

```
ALTER TABLE instructor MODIFY dept_name NOT NULL;
```

This works **only if no existing row has NULL** in that column. Fix the nulls first with an **UPDATE**, otherwise Oracle raises an error.

Syntax across databases

Operation	Oracle	MySQL	PostgreSQL
Add column	<code>ADD col type</code>	<code>ADD col type</code>	<code>ADD COLUMN col type</code>

Operation	Oracle	MySQL	PostgreSQL
Drop column	<code>DROP COLUMN col</code>	<code>DROP COLUMN col</code>	<code>DROP COLUMN col</code>
Rename column	<code>RENAME COLUMN a TO b</code>	<code>RENAME COLUMN a TO b</code>	<code>RENAME COLUMN a TO b</code>
Modify data type	<code>MODIFY col newtype</code>	<code>MODIFY col newtype</code>	<code>ALTER COLUMN col TYPE newtype</code>
Add NOT NULL	<code>MODIFY col NOT NULL</code>	<code>MODIFY col type NOT NULL</code>	<code>ALTER COLUMN col SET NOT NULL</code>

Related commands: DROP, TRUNCATE, DELETE

Command	What it removes	Can undo?	Example
<code>DROP TABLE t;</code>	The whole table (structure + data)	No	<code>DROP TABLE teaches;</code>
<code>TRUNCATE TABLE t;</code>	All rows , keeps the structure	No	<code>TRUNCATE TABLE teaches;</code>
<code>DELETE FROM t WHERE ...;</code>	Selected rows	Yes (before COMMIT)	<code>DELETE FROM teaches WHERE year = 2009;</code>

3.5 Basic Query Structure: SELECT, FROM, WHERE [FOUNDATION: Background Concept]

Every basic SQL query has three clauses:

```
SELECT  what to show      (columns)      <- like choosing columns
FROM    where to look     (tables)       <- like choosing the sheet
WHERE   which rows to keep (condition)   <- like a filter
```

```
SELECT name, salary
FROM   instructor
WHERE  dept_name = 'Comp. Sci.' AND salary > 70000;
```

Result:

```
+-----+-----+
| name  | salary |
+-----+-----+
| Katz   | 75000  |
| Brandt | 92000  |
+-----+-----+
(Srinivasan earns 65000, so he is filtered out)
```

Handy clauses:

Clause	Purpose	Example	Result
<code>DISTINCT</code>	Remove duplicate rows	<code>SELECT DISTINCT dept_name FROM instructor;</code>	Six departments, Comp. Sci. only once
<code>*</code>	All columns	<code>SELECT * FROM instructor;</code>	The whole table
<code>AS</code>	Rename a column in output	<code>SELECT salary/12 AS monthly FROM instructor;</code>	Monthly pay
<code>ORDER BY</code>	Sort the result	<code>SELECT name, salary FROM instructor ORDER BY salary DESC;</code>	Einstein (95000) first
<code>LIKE</code>	Pattern match (<code>%</code> = any text, <code>_</code> = one character)	<code>WHERE name LIKE 'S%'</code>	Srinivasan
<code>BETWEEN</code>	Range test	<code>WHERE salary BETWEEN 60000 AND 80000</code>	Srinivasan, El Said, Katz, Crick

Order in which SQL processes a query (very useful for HAVING later):

```
FROM  -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY
(tables) (filter (make (filter (choose (sort)
rows) groups) groups) columns)
```

3.6 Set Operations: UNION, INTERSECT, EXCEPT [🔥 HIGH-YIELD: Frequently Asked]

Set operations **combine the results of two queries**. Asked in both 2022 papers (6 marks each).

Analogy: Think of two guest lists for two parties. **UNION** = everyone invited to either party. **INTERSECT** = people invited to both. **EXCEPT** = people on list 1 who are **not** on list 2.

Rules: both queries must return the **same number of columns** with **compatible data types**.

Assumed data (courses offered in two semesters, from the `section` table):

```
Fall 2009 courses : CS-101, CS-347, PHY-101
Spring 2010 courses: CS-101, CS-315, CS-319, FIN-201
```

Fall only	BOTH	Spring only
CS-347	CS-101	CS-315
PHY-101		CS-319
		FIN-201

UNION = all three columns together (6 courses)
 INTERSECT = the middle column only (1 course)
 EXCEPT = Fall only (first minus second) (2 courses)

UNION: everything from both, duplicates removed

```
SELECT course_id FROM section WHERE semester = 'Fall' AND year = 2009
UNION
SELECT course_id FROM section WHERE semester = 'Spring' AND year = 2010;
```

Result (6 rows): CS-101, CS-347, PHY-101, CS-315, CS-319, FIN-201. CS-101 appears **once**.

Keep duplicates with `UNION ALL`: CS-101 would appear **twice** (7 rows).

INTERSECT: only rows present in both

```
SELECT course_id FROM section WHERE semester = 'Fall' AND year = 2009
INTERSECT
SELECT course_id FROM section WHERE semester = 'Spring' AND year = 2010;
```

Result (1 row): CS-101.

EXCEPT: rows in the first query but not in the second

```
SELECT course_id FROM section WHERE semester = 'Fall' AND year = 2009
EXCEPT
SELECT course_id FROM section WHERE semester = 'Spring' AND year = 2010;
```

Result (2 rows): CS-347, PHY-101 (taught in Fall 2009 but not in Spring 2010).

Oracle note: Oracle uses `MINUS` instead of `EXCEPT`. The textbook and PostgreSQL use `EXCEPT`. Mention both in the exam.

Summary

Operator	Meaning	Duplicates	Venn picture	Result from the example
UNION	A or B	Removed (use <code>UNION ALL</code> to keep)	Both circles	6 courses
INTERSECT	A and B	Removed (use <code>INTERSECT ALL</code> to keep)	Overlap only	CS-101
EXCEPT / MINUS	A but not B	Removed (use <code>EXCEPT ALL</code> to keep)	A without the overlap	CS-347, PHY-101

Note: the order matters for **EXCEPT** only. **A EXCEPT B** is different from **B EXCEPT A**.

3.7 Joins HIGH-YIELD: Frequently Asked]

A **join** combines columns from **two or more tables** based on a related column.

Analogy: You have a class register (name, department code) and a separate list of department codes with department names. A join **stitches** the two lists into one, matching on the code.

Why joins? A database splits data into many tables to avoid repetition. Joins bring it back together when you need a full answer.

Tables used for every join example

EMPLOYEES			DEPARTMENTS	
employee_id	first_name	dept_id	department_id	department_name
1	Asha	10	10	HR
2	Ravi	20	20	IT
3	Meena	10	30	Finance
4	John	NULL		

(John has no department. Finance has no employees.)

Which rows does each join keep?

	A-only rows	matching rows	B-only rows
INNER JOIN	--	YES	--
LEFT JOIN	YES	YES	--
RIGHT JOIN	--	YES	YES
FULL OUTER JOIN	YES	YES	YES
CROSS JOIN	every row of A paired with every row of B (no condition)		

(A = left table = employees, B = right table = departments)

1. INNER JOIN: only matching rows

```
SELECT e.employee_id, e.first_name, d.department_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.department_id;
```

employee_id	first_name	department_name
1	Asha	HR
2	Ravi	IT
3	Meena	HR

3 rows

John (no department) and Finance (no employees) are **both dropped**. **JOIN** alone means **INNER JOIN**.

2. LEFT (OUTER) JOIN: all rows from the left table

```
SELECT e.employee_id, e.first_name, d.department_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.department_id;
```

employee_id	first_name	department_name
1	Asha	HR
2	Ravi	IT
3	Meena	HR
4	John	NULL

<- kept, no match
4 rows

Use it for: "show all employees, even those without a department". Add `WHERE d.department_id IS NULL` to list **only** the unmatched ones (John).

3. RIGHT (OUTER) JOIN: all rows from the right table

```
SELECT e.employee_id, e.first_name, d.department_name
FROM   employees e
RIGHT JOIN departments d ON e.dept_id = d.department_id;
```

employee_id	first_name	department_name
1	Asha	HR
3	Meena	HR
2	Ravi	IT
NULL	NULL	Finance

<- kept, no match
4 rows

Use it for: "show all departments, even empty ones".

4. FULL OUTER JOIN: all rows from both tables

```
SELECT e.employee_id, e.first_name, d.department_name
FROM   employees e
FULL OUTER JOIN departments d ON e.dept_id = d.department_id;
```

employee_id	first_name	department_name
1	Asha	HR
2	Ravi	IT
3	Meena	HR
4	John	NULL
NULL	NULL	Finance

<- left-only row
<- right-only row
5 rows

Use it for: a complete picture; unmatched rows from **either** side show up with NULLs.

5. CROSS JOIN: every possible pair (Cartesian product)

```
SELECT e.first_name, d.department_name
FROM   employees e
CROSS JOIN departments d;
```

```
Asha-HR, Asha-IT, Asha-Finance,
Ravi-HR, Ravi-IT, Ravi-Finance,
Meena-HR, Meena-IT, Meena-Finance,
John-HR, John-IT, John-Finance      = 4 x 3 = 12 rows
```

There is **no ON condition**. The row count is **rows(A) x rows(B)**.

Analogy: A menu with 4 starters and 3 mains. A cross join lists **all 12 possible meal combinations**, whether sensible or not.

Use it for: generating all combinations (e.g., every size with every colour of a T-shirt). Be careful with big tables: 1,000 x 1,000 rows gives 1,000,000 rows.

6. NATURAL JOIN (bonus)

A **NATURAL JOIN** automatically joins on the columns that have the **same name** in both tables.

```
SELECT * FROM instructor NATURAL JOIN teaches; -- matches on the common column ID
```

Comparison summary

Join	Rows returned from the employees/departments example	Unmatched rows kept?
INNER	3	None
LEFT	4	From left only

Join	Rows returned from the employees/departments example	Unmatched rows kept?
RIGHT	4	From right only
FULL OUTER	5	From both
CROSS	12	Not applicable (no matching at all)

Exam Tip: For each join write: definition, syntax, a small result table, and a one-line use case. For the Jan-25 style question (full outer + cross), also state the row counts (5 and 12). A Venn-style line like the "which rows does each join keep" diagram earns presentation marks.

3.8 Aggregate Functions, GROUP BY and HAVING [★ ESSENTIAL FOR EXAM]

Aggregate functions take many rows and return one value.

Function	Meaning	Example on <code>instructor</code>	Result
<code>COUNT(*)</code>	Number of rows	<code>SELECT COUNT(*) FROM instructor;</code>	8
<code>SUM(col)</code>	Total	<code>SELECT SUM(salary) FROM instructor;</code>	589000
<code>AVG(col)</code>	Average	<code>SELECT AVG(salary) FROM instructor;</code>	73625
<code>MIN(col)</code>	Smallest	<code>SELECT MIN(salary) FROM instructor;</code>	40000
<code>MAX(col)</code>	Largest	<code>SELECT MAX(salary) FROM instructor;</code>	95000

GROUP BY: one result per group

Analogy: Sort the exam answer sheets into piles by class division, then compute the average of each pile.

```
SELECT dept_name, AVG(salary) AS avg_salary, COUNT(*) AS no_of_instructors
FROM instructor
GROUP BY dept_name;
```

```
+-----+-----+-----+
| dept_name | avg_salary | no_of_instructors |
+-----+-----+-----+
| Comp. Sci. | 77333.33 | 3 | (65000+75000+92000)/3
| Finance | 90000 | 1 |
| Music | 40000 | 1 |
| Physics | 95000 | 1 |
| History | 60000 | 1 |
| Biology | 72000 | 1 |
+-----+-----+-----+
```

Rule: every column in `SELECT` that is **not** inside an aggregate function must appear in `GROUP BY`.

HAVING: filter the groups

`HAVING` is a filter that works on groups (after `GROUP BY`).

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 42000;
```

```
+-----+-----+
| Comp. Sci. | 77333.33 |
| Finance | 90000 |
| Physics | 95000 |
| History | 60000 |
| Biology | 72000 |
+-----+-----+
(Music, average 40000, is removed)
```

WHERE vs HAVING (a classic exam question)

	WHERE	HAVING
Filters	Individual rows	Groups of rows
Runs	Before GROUP BY	After GROUP BY
Can use aggregate functions?	No	Yes
Example	WHERE salary > 60000	HAVING AVG(salary) > 60000

```
-- Using both: only instructors above 60000 are counted, then only large groups kept
SELECT dept_name, COUNT(*) AS n
FROM instructor
WHERE salary > 60000
GROUP BY dept_name
HAVING COUNT(*) >= 2;
-- Result: Comp. Sci. | 3 (Srinivasan 65000, Katz 75000, Brandt 92000)
```

Null rule: all aggregate functions **except** `COUNT(*)` ignore NULL values.

3.9 Nested Subqueries: IN, EXISTS, SOME, ALL [★ ESSENTIAL FOR EXAM]

A **subquery** is a query **inside another query**. The inner query runs and gives its answer to the outer one.

Analogy: "Find the students who scored above the class average." First compute the class average (inner query), then use it to pick students (outer query).

IN / NOT IN: set membership

```
-- Courses taught in Fall 2009 that are ALSO taught in Spring 2010
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND year = 2009
AND course_id IN (SELECT course_id FROM section
WHERE semester = 'Spring' AND year = 2010);
-- Result: CS-101
```

Use `NOT IN` for "taught in Fall 2009 but **not** in Spring 2010" (result: CS-347, PHY-101).

EXISTS / NOT EXISTS: "is there at least one matching row?"

`EXISTS` returns **TRUE** if the subquery returns **at least one row**. The inner query refers to a column of the outer query (a **correlated** subquery).

```
SELECT course_id
FROM section S
WHERE semester = 'Fall' AND year = 2009
AND EXISTS (SELECT * FROM section T
WHERE T.semester = 'Spring' AND T.year = 2010
AND T.course_id = S.course_id);
```

```
How it works, row by row (outer rows from Fall 2009):
S = CS-101 -> is there a Spring 2010 row with CS-101? YES -> keep
S = CS-347 -> ... with CS-347? NO -> drop
S = PHY-101 -> ... with PHY-101? NO -> drop
Result: CS-101
```

Analogy: `EXISTS` is a **door-bell test**. "Is anybody home?" You do not care who or how many, only whether the answer is yes or no.

SOME (ANY) / ALL: comparing with a set

```
-- Salary greater than AT LEAST ONE Comp. Sci. salary (65000, 75000, 92000)
SELECT name FROM instructor
WHERE salary > SOME (SELECT salary FROM instructor WHERE dept_name = 'Comp. Sci.');
```

-- Result: Wu, Einstein, Katz, Crick, Brandt (anyone above 65000)

```
-- Salary greater than EVERY Comp. Sci. salary
SELECT name FROM instructor
WHERE salary > ALL (SELECT salary FROM instructor WHERE dept_name = 'Comp. Sci.');
```

-- Result: Einstein (only 95000 beats 92000)

Operator	Meaning
IN	Equal to any value in the set (same as = SOME)
NOT IN	Not equal to any value in the set
EXISTS	Subquery returns at least one row
NOT EXISTS	Subquery returns no rows
> SOME	Greater than at least one value
> ALL	Greater than every value

3.10 Modifying Data: INSERT, UPDATE, DELETE [💡 POSSIBLE SURPRISE: In Syllabus, Not Yet Asked]

```
-- INSERT: add a new row
INSERT INTO course VALUES ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
INSERT INTO course (title, course_id, credits, dept_name) -- order may change
VALUES ('Robotics', 'CS-501', 3, 'Comp. Sci.');
```

```
-- UPDATE: change existing rows
UPDATE instructor SET salary = salary * 1.05 WHERE salary < 70000; -- 5% raise
```

```
-- DELETE: remove rows
DELETE FROM instructor WHERE dept_name = 'Finance';
DELETE FROM instructor WHERE salary BETWEEN 13000 AND 15000;
```

Remember: **UPDATE** and **DELETE** without a **WHERE** affect every row. Use **COMMIT** to save and **ROLLBACK** to undo.

Statement	Without WHERE	With WHERE
UPDATE	Changes all rows	Changes only matching rows
DELETE	Deletes all rows (structure stays)	Deletes only matching rows

Unit 4: Normalization

4.1 What is Normalization and Why is it Important? [🔥 HIGH-YIELD: Frequently Asked]

Normalization was asked in **all four** past papers, always as "why is normalization important? Discuss 1NF, 2NF and 3NF with examples".

Definition: Normalization is the process of **organising data into well-structured tables** so that **redundancy (repetition) is minimised** and data stays **consistent**. It is done by **splitting a big table into smaller related tables** and linking them with keys.

Analogy: A messy cupboard where socks, shirts and papers are mixed in the same drawers. Normalization is **sorting every item into its own proper drawer**, so each item has exactly one place. Finding, changing and removing things becomes easy, and nothing is stored twice.

Why is normalization important?

- **Reduces redundancy:** the same fact is stored only once, saving space.
- **Prevents anomalies:** removes insertion, update and deletion problems (shown below).
- **Improves consistency:** one fact in one place cannot contradict itself.
- **Makes the design cleaner:** each table describes one thing.
- **Easier maintenance:** changes happen in one place.

The three anomalies (what goes wrong without normalization)

Assumed data: one badly designed table `STUDENT_COURSE`.

```
STUDENT_COURSE
+-----+-----+-----+-----+-----+
| roll_no | name  | course_id | course_name | instructor |
+-----+-----+-----+-----+-----+
| 1       | Asha  | C1       | DBMS       | Dr. Rao    |
| 1       | Asha  | C2       | Java       | Dr. Shah   |
| 2       | Ravi  | C1       | DBMS       | Dr. Rao    |
| 3       | Meena | C3       | Networks   | Dr. Iyer   |
+-----+-----+-----+-----+-----+
Primary key = (roll_no, course_id)
```

<- DBMS and Dr. Rao repeated

Anomaly	What happens	Example in the table above
Insertion anomaly	You cannot add some data without adding other, unrelated data	A new course C4 "Python" cannot be added until some student enrolls, because <code>roll_no</code> is part of the key and cannot be NULL
Update anomaly	A change must be made in many rows; missing one creates inconsistency	If Dr. Rao is replaced by Dr. Mehta, rows 1 and 3 must both change. Updating only one leaves two different instructors for DBMS
Deletion anomaly	Deleting one fact accidentally deletes another	Meena is the only student of C3. Deleting her row also erases the fact that course C3 "Networks" taught by Dr. Iyer exists

4.2 Functional Dependency: The Idea Behind Every Normal Form [📌]

FOUNDATION: Background Concept]

Functional dependency (FD): $X \rightarrow Y$ means "if you know X, you can determine exactly one value of Y". Read it as "X determines Y".

Analogy: If you know a person's **Aadhaar number**, you know their **name**. So Aadhaar $\rightarrow$ name. The reverse is not true; many people can share a name.

Examples:

- `roll_no -> name` (a roll number belongs to exactly one student).
- `course_id -> course_name, instructor` (each course has one name and one instructor).
- `emp_zip -> emp_city, emp_state` (a pincode belongs to one city).

Key words you must know:

Term	Meaning	Example (table <code>STUDENT</code> , <code>COURSE</code>)
Prime attribute	Part of a candidate key	<code>roll_no</code> , <code>course_id</code>
Non-prime attribute	Not part of any candidate key	<code>name</code> , <code>course_name</code> , <code>instructor</code>
Partial dependency	A non-prime attribute depends on only part of a composite key	<code>roll_no -> name</code> (name needs only <code>roll_no</code> , not the whole key)
Transitive dependency	A non-prime attribute depends on another non-prime attribute: <code>A -> B -> C</code>	<code>emp_id -> emp_zip -> emp_city</code>

THE NORMALIZATION LADDER (each step needs the previous one)

```

Unnormalized table (messy, repeating groups)
| rule: every cell holds ONE value
v
+-----+
| 1NF atomic values |
+-----+
| rule: remove PARTIAL dependencies
v
+-----+
| 2NF |
+-----+
| rule: remove TRANSITIVE dependencies
v
+-----+
| 3NF |
+-----+
| rule: every determinant must be a super key
v
+-----+
| BCNF (stronger 3NF) |
+-----+

```

4.3 First Normal Form (1NF) [🔥 HIGH-YIELD: Frequently Asked]

Rule: a table is in **1NF** if **every column holds a single atomic (indivisible) value**: no lists, no repeating groups, no multi-valued cells.

Analogy: Each box on a form takes **one answer only**. If someone writes two phone numbers in the one box, you cannot sort or search properly.

Example: violates 1NF (the `phone` column holds two values for Ron):

```

STUDENT (not in 1NF)
+-----+-----+-----+-----+
| stu_id | name  | city  | phone                |
+-----+-----+-----+-----+
| 101    | Asha  | Surat | 9876500001           |
| 102    | Ron   | Chennai | 7778881212, 9990000123 |  <- two values!
| 103    | Lester | Bangalore | 9990000555           |
+-----+-----+-----+-----+

```

Step-by-step conversion:

1. Find the column with multiple values (`phone`).
2. Give each value **its own row** (repeat the other columns), or move phones to a separate table.
3. Choose a primary key for the new table.

After (in 1NF):

```

STUDENT (1NF)
+-----+-----+-----+-----+
| stu_id | name  | city  | phone |
+-----+-----+-----+-----+
| 101    | Asha  | Surat | 9876500001 |
| 102    | Ron   | Chennai | 7778881212 |
| 102    | Ron   | Chennai | 9990000123 |    <- one value per cell
| 103    | Lester | Bangalore | 9990000555 |
+-----+-----+-----+-----+
Primary key = (stu_id, phone)

```

Another way (cleaner): split into two tables so the name and city are not repeated:

```

STUDENT(stu_id, name, city)    STUDENT_PHONE(stu_id, phone)

```

4.4 Second Normal Form (2NF) [🔥 HIGH-YIELD: Frequently Asked]

Rule: a table is in **2NF** if (1) it is in **1NF** and (2) it has **no partial dependency**: every non-prime attribute depends on the **whole** candidate key, not on just part of it.

Analogy: In a restaurant bill with key (table_no, dish): the **price of a dish** depends only on the dish, not on which table ordered it. Keeping the price in the bill repeats it for every table, so it belongs in a separate menu list.

Note: if the primary key is a **single column**, a 1NF table is automatically in 2NF. The 2NF check matters only for **composite keys**.

Example: violates 2NF

```

TEACHER_SUBJECT
+-----+-----+-----+
| teacher_id | subject | teacher_age |
+-----+-----+-----+
| 111        | Maths   | 38           |
| 111        | Physics | 38           |    <- age of 111 repeated
| 222        | Biology | 38           |
| 333        | Physics | 40           |
| 333        | Chemistry | 40          |
+-----+-----+-----+
Candidate key = (teacher_id, subject)
Problem: teacher_age depends only on teacher_id (PART of the key)
        teacher_id -> teacher_age    <- partial dependency

```

Step-by-step conversion:

1. List the candidate key: (teacher_id, subject).
2. Find non-prime attributes that depend on only part of it: teacher_age depends on teacher_id.
3. Move that attribute and its determinant into a **new table**.
4. Keep the original table with the full key.

After (in 2NF):

```

TEACHER_DETAILS          TEACHER_SUBJECT
+-----+-----+          +-----+-----+
| teacher_id | teacher_age |          | teacher_id | subject |
+-----+-----+          +-----+-----+
| 111        | 38          |          | 111        | Maths   |
| 222        | 38          |          | 111        | Physics |
| 333        | 40          |          | 222        | Biology |
+-----+-----+          | 333        | Physics |
| (age stored ONCE per teacher) |          | 333        | Chemistry |
+-----+-----+          +-----+-----+

```

If teacher 111's age changes, you update **one** row.

4.5 Third Normal Form (3NF) [🔥 HIGH-YIELD: Frequently Asked]

Rule: a table is in **3NF** if (1) it is in **2NF** and (2) it has **no transitive dependency**: no non-prime attribute depends on another non-prime attribute.

Simple memory line: "Every non-key attribute must depend on **the key, the whole key, and nothing but the key.**"

Analogy: A school ID card shows **pincode**, **city** and **state**. The city and state are decided by the **pincode**, not by the student. Storing them for every student repeats the same pincode-to-city fact thousands of times. Keep a separate pincode list.

Example: violates 3NF

```

EMPLOYEE
+-----+-----+-----+-----+-----+
| emp_id | emp_name | emp_zip | emp_state | emp_city |
+-----+-----+-----+-----+-----+
| 1001 | John | 282005 | UP | Agra |
| 1002 | Ajeet | 222008 | UP | Mau |
| 1006 | Lora | 282007 | UP | Agra |
| 1101 | Lilly | 292008 | UP | Pratapgarh |
| 1201 | Steve | 222999 | UP | Mau |
+-----+-----+-----+-----+-----+

Candidate key = emp_id
emp_id -> emp_zip (fine)
emp_zip -> emp_state, emp_city
So emp_id -> emp_zip -> emp_city = TRANSITIVE dependency
  
```

<- Agra / UP repeated

Step-by-step conversion:

1. Find the chain **A -> B -> C** where B and C are non-key: **emp_id -> emp_zip -> emp_city**.
2. Move **B** and **C** (the dependent attributes) into a new table keyed by **B** (**emp_zip**).
3. Keep **B** in the original table as a **foreign key**.

After (in 3NF):

```

EMPLOYEE                                EMPLOYEE_ZIP
+-----+-----+-----+-----+-----+
| emp_id | emp_name | emp_zip |      | emp_zip | emp_state | emp_city |
+-----+-----+-----+-----+-----+
| 1001 | John | 282005 | ----->| 282005 | UP | Agra |
| 1002 | Ajeet | 222008 |      | 222008 | UP | Mau |
| 1006 | Lora | 282007 |      | 282007 | UP | Agra |
| 1101 | Lilly | 292008 |      | 292008 | UP | Pratapgarh |
| 1201 | Steve | 222999 |      | 222999 | UP | Mau |
+-----+-----+-----+-----+-----+
(emp_zip in EMPLOYEE is a foreign key to EMPLOYEE_ZIP)
  
```

4.6 One Complete Example: UNF to 1NF to 2NF to 3NF [🔥 HIGH-YIELD: Frequently Asked]

Examiners like a **single example taken through all the forms**. Use this one if the question says "with appropriate examples".

Assumed data: employees working on projects.

Step 0: Unnormalized table (UNF): one cell contains several values.

```

+-----+-----+-----+-----+-----+-----+
| emp_id | emp_name | zip | city | project_id | hours |
+-----+-----+-----+-----+-----+-----+
| 101 | Asha | 380001 | Ahmedabad | P1, P2 | 10, 20 |
| 102 | Ravi | 395001 | Surat | P1 | 15 |
+-----+-----+-----+-----+-----+-----+
  
```

<- lists

Step 1: Convert to 1NF: one value per cell. Key becomes (**emp_id**, **project_id**).

emp_id	emp_name	zip	city	project_id	hours
101	Asha	380001	Ahmedabad	P1	10
101	Asha	380001	Ahmedabad	P2	20
102	Ravi	395001	Surat	P1	15

FDs: emp_id -> emp_name, zip, city (depends on PART of the key)
 (emp_id, project_id) -> hours (depends on the WHOLE key)
 zip -> city

Step 2: Convert to 2NF: remove the partial dependency ($\text{emp_id} \rightarrow \text{name, zip, city}$).

EMPLOYEE				WORKS_ON		
emp_id	emp_name	zip	city	emp_id	project_id	hours
101	Asha	380001	Ahmedabad	101	P1	10
102	Ravi	395001	Surat	101	P2	20
				102	P1	15

key = emp_id

key = (emp id, project id)

Step 3: Convert to 3NF: remove the transitive dependency ($\text{emp id} \rightarrow \text{zip} \rightarrow \text{city}$).

EMPLOYEE			ZIP_CITY		WORKS_ON		
emp_id	emp_name	zip	zip	city	emp_id	project_id	hours
101	Asha	380001	380001	Ahmedabad	101	P1	10
102	Ravi	395001	395001	Surat	101	P2	20
					102	P1	15

Result: three small tables with **no repeated facts**. The anomalies are gone: a city name is changed in one place; a new zip can be added without an employee.

4.7 Boyce-Codd Normal Form (BCNF) [💡 POSSIBLE SURPRISE: In Syllabus, Not Yet Asked]

Rule: a table is in **BCNF** if, for **every** functional dependency $X \rightarrow Y$, **X is a super key**. It is a **stronger** version of 3NF (sometimes called "3.5NF"). Your syllabus ends with BCNF.

Relation: every BCNF relation is in 3NF, but not every 3NF relation is in BCNF.

Analogy: 3NF allows a small loophole (a dependency whose right side is part of a key). BCNF closes the loophole: **anything that decides something must be a key.**

Example: ADVISOR(student, subject, professor).

- Each professor teaches **only one subject**: `professor -> subject`.
- A student has **one professor per subject**: `(student, subject) -> professor`.
- Candidate keys: `(student, subject)` and `(student, professor)`.

```
ADVISOR
+-----+-----+-----+
| student | subject | professor |
+-----+-----+-----+
| Asha    | Maths   | Prof. A   |
| Ravi    | Maths   | Prof. A   |
| Asha    | Physics | Prof. B   |
+-----+-----+-----+
professor -> subject, but professor is NOT a super key => violates BCNF
(it is still in 3NF because "subject" is a prime attribute)
```

Decompose:

PROF_SUBJECT			STUDENT_PROF		
professor	subject		student	professor	
Prof. A	Maths		Asha	Prof. A	
Prof. B	Physics		Ravi	Prof. A	
			Asha	Prof. B	
professor is the key					

Now every determinant (**professor**) is a key in its table, so both tables are in BCNF.

4.8 Normal Forms at a Glance [🔥 HIGH-YIELD: Frequently Asked]

Normal form	Rule in one line	Removes	Quick check question
1NF	One atomic value per cell	Repeating groups, multi-valued cells	"Is any cell holding a list?"
2NF	1NF + no partial dependency	Dependence on part of a composite key	"Does any column depend on only part of the key?"
3NF	2NF + no transitive dependency	Non-key depending on non-key	"Does any non-key column decide another non-key column?"
BCNF	Every determinant is a super key	Remaining overlap problems	"Is every left-hand side of an FD a key?"

```

Table has repeating / multi-valued cells?    --yes--> make it 1NF
| no
Composite key AND column depends on part of it? --yes--> make it 2NF
| no
Non-key column determines another non-key column? --yes--> make it 3NF
| no
Some determinant is not a super key?          --yes--> make it BCNF
| no
DONE: the design is well normalized

```

Trade-off to mention for extra marks: too much normalization means **more tables and more joins**, which can slow queries. Some systems **denormalize** (add controlled redundancy) for speed in reports.

Exam Tip (7-8 marks): (1) Define normalization and give 3-4 reasons it is important (1 mark each). (2) Show the 3 anomalies briefly. (3) Take one example through 1NF, 2NF and 3NF with before/after tables (about 1.5 marks per form). (4) Add BCNF in 2-3 lines as a bonus.

Quick Revision Section

A. Past-Paper Question Bank with Answer Locations

Practise writing these from memory. They are the questions that actually appeared.

#	Question (shortened)	Marks	Asked in	Answer in these notes
1	Write SQL to create the university schema (department, course, instructor, section, teaches)	9	Jan-22, Feb-22, Jan-23	Section 3.3
2	Explain ALTER TABLE: add, drop, rename column, modify datatype, add NOT NULL	8	Jan-25	Section 3.4
3	Why normalization? Discuss 1NF, 2NF, 3NF with examples	7-8	All four papers	Unit 4 (4.1 to 4.6)
4	Why SQL keys? Super, candidate, composite, alternate (and artificial) keys	6-8	All four papers	Section 2.3
5	Explain referential integrity with example	6-8	Jan-23, Jan-25	Section 2.4
6	Union, intersect, except with SQL examples	6	Jan-22, Feb-22	Section 3.6
7	Inner, left, right, full outer join (+ HAVING, EXISTS)	6	Jan-23	Sections 3.7, 3.8, 3.9
8	Full outer join and cross join with employees and departments	8	Jan-25	Section 3.7
9	Roles and functions of the DBA	8-9	Jan-23, Jan-25	Section 1.9
10	What is a data model? Categories of data models	6	Jan-22, Feb-22	Section 1.6
11	File-processing system: pros and cons	6	Jan-23	Section 1.3
12	What is a DBMS? Primary goal?	6	Jan-23	Section 1.1
13	Components of a database system with diagram	7	Feb-22	Section 1.8
14	ER model and mapping cardinality	7	Jan-23	Section 1.10

B. Answer-Writing Templates

Use these skeletons so no mark is missed.

Template for "Discuss / Explain X with example" (6-7 marks)

1. **Definition** (1 mark), in one or two lines.
2. **Why it is needed or its purpose** (1 mark).
3. **Types / features / steps**, each with a short explanation (2-3 marks).
4. **Example with small table or code** (1-2 marks). Begin with "Assumed data:".
5. **Diagram** if it fits (1 mark).

Template for "Write SQL" questions (9 marks)

1. **Assumptions** line (data types, keys).
2. **Correct order** of statements.
3. **Primary key and foreign key** on every table.
4. **Constraints** (NOT NULL, CHECK) where sensible.
5. **One-line reason** for key choices.

Template for "Normalization" (7-8 marks)

1. Definition + importance (2 marks).
2. Anomalies in a single table (1 mark).
3. 1NF with before/after (1.5 marks).
4. 2NF with before/after (1.5 marks).
5. 3NF with before/after (1.5 marks).
6. BCNF mention (0.5 mark bonus).

C. One-Page Cheat Sheet

Definitions in one line

Term	One-line definition
DBMS	Interrelated data plus programs to access it, convenient and efficient
Schema / Instance	Design of the database / data in it at one moment
Data model	Tools to describe data, relationships, semantics and constraints
DDL / DML	Defines structure / works with data
DBA	Person with central control of the database
Super key	Any attribute set that uniquely identifies a row
Candidate key	Minimal super key
Primary key	The chosen candidate key (unique, not null, only one)
Alternate key	Candidate keys not chosen as primary
Foreign key	Attribute referring to another table's primary key
Composite key	Key of two or more attributes
Artificial key	System-generated key with no real-world meaning
Referential integrity	Foreign key must be NULL or match an existing parent key
Normalization	Splitting tables to remove redundancy and anomalies
1NF / 2NF / 3NF / BCNF	Atomic / no partial dependency / no transitive dependency / every determinant a key

SQL syntax in one place

```
-- CREATE
CREATE TABLE t (c1 NUMBER(5) PRIMARY KEY, c2 VARCHAR2(20) NOT NULL,
                 c3 NUMBER(5), FOREIGN KEY (c3) REFERENCES other(id));

-- ALTER (Oracle)
ALTER TABLE t ADD c4 VARCHAR2(10);
ALTER TABLE t DROP COLUMN c4;
ALTER TABLE t RENAME COLUMN c2 TO c2_new;
ALTER TABLE t MODIFY c2_new VARCHAR2(40);
ALTER TABLE t MODIFY c3 NOT NULL;

-- SET OPERATIONS (Oracle: MINUS instead of EXCEPT)
SELECT x FROM a UNION      SELECT x FROM b;
SELECT x FROM a INTERSECT  SELECT x FROM b;
SELECT x FROM a EXCEPT   SELECT x FROM b;

-- JOINS
SELECT ... FROM a INNER JOIN      b ON a.k = b.k;
SELECT ... FROM a LEFT JOIN       b ON a.k = b.k;
SELECT ... FROM a RIGHT JOIN      b ON a.k = b.k;
SELECT ... FROM a FULL OUTER JOIN b ON a.k = b.k;
SELECT ... FROM a CROSS JOIN      b;

-- GROUP BY / HAVING / EXISTS
SELECT d, AVG(s) FROM t GROUP BY d HAVING AVG(s) > 50000;
SELECT * FROM a WHERE EXISTS (SELECT 1 FROM b WHERE b.k = a.k);
```

Mnemonics

- **File-processing problems:** R-A-D-I-A-C-S = Redundancy, Access, Data isolation, Integrity, Atomicity, Concurrent access, Security.
- **Key hierarchy:** Super key contains Candidate key contains Primary key. Alternate = Candidate minus Primary.
- **Normal forms:** "1NF: one value. 2NF: whole key. 3NF: nothing but the key."
- **Joins:** Inner = overlap. Left = all left. Right = all right. Full = everything. Cross = every pair.
- **WHERE vs HAVING:** WHERE filters **rows before** grouping. HAVING filters **groups after** grouping.
- **Order of query processing:** FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY.

D. Last-Minute Checklist

- ☐ Can I write the five-table university schema from memory, with composite keys in **section** and **teaches**?
- ☐ Can I write all five ALTER TABLE forms?
- ☐ Can I take one table through 1NF, 2NF and 3NF with before/after tables?
- ☐ Can I define all seven key types using one example table?
- ☐ Can I explain referential integrity with a rejected-insert example?
- ☐ Can I draw the five join outputs for two small tables (3, 4, 4, 5 and 12 rows)?
- ☐ Can I list the five DBA functions with one example each?
- ☐ Can I list the seven file-processing disadvantages?
- ☐ Can I name the four data-model categories with an example each?
- ☐ Have I drawn the database-system components diagram and the 3-tier diagram once?

Final advice: The top five topics are about 80% of the sessional marks in past papers. If time is short, master Sections 3.3, 3.4, Unit 4, Section 2.3, Sections 3.6 and 3.7, and Section 1.9 first. Then read the remaining sections once for definitions and examples.